

ETL & Pipelines

Apache Airflow

Comprendre les pipelines de données et les orchestrer avec Airflow.
DAGs, Tasks, Operators, Scheduling & Monitoring.

AniData Lab — Séances 4-8 — Mercredi 25 → Vendredi 27 mars 2026

Objectifs de ce module

Ce que vous maîtriserez à la fin de la semaine.

1

Comprendre ETL vs ELT

Définir les patterns, savoir quand les utiliser.

2

Maîtriser les concepts Airflow

DAG, Task, Operator, XCom, Schedule.

3

Créer des DAGs fonctionnels

De l'extraction à l'indexation dans ES.

4

Orchestrer un pipeline complet

Extract → Transform → Load automatisé.

5

Monitorer et débbugger

Logs, retries, alertes, bonnes pratiques.



CHAPITRE 1

ETL & ELT

Les deux grands patterns d'intégration de données.

C'est quoi un pipeline de données ?

Un pipeline est une chaîne de traitements automatisés appliqués séquentiellement aux données.

Imaginez une chaîne de montage dans une usine : chaque poste reçoit un produit semi-fini, effectue une opération, et le passe au poste suivant. Un pipeline de données fonctionne exactement pareil : chaque étape reçoit des données, les transforme, et les transmet.



ETL : Extract → Transform → Load

Le pattern classique : on transforme AVANT de charger dans la cible.

1. EXTRACT

Extraire les données des sources :

- Fichiers (CSV, JSON, XML)
- Bases de données (SQL)
- APIs (REST, GraphQL)
- Streams (Kafka, logs)



2. TRANSFORM

Transformer en mémoire :

- Nettoyage (NaN, doublons)
- Normalisation (types, formats)
- Enrichissement (features)
- Validation (assertions)



3. LOAD

Charger dans la cible :

- Data warehouse
- Elasticsearch
- Base de production
- Fichier exporté

C'est exactement ce qu'on a fait hier matin : scripts 03 (Extract+Transform) → 05 (Validate) → 06 (Load dans ES) !

ELT : Extract → Load → Transform

Le pattern moderne : on charge d'abord les données brutes, puis on les transforme sur place.

1. EXTRACT

Extraire les données
des sources.

Identique à l'ETL.



2. LOAD

Charger les données
BRUTES dans la cible.

Pas de transformation
avant le chargement.



3. TRANSFORM

Transformer DANS
la cible (SQL, dbt).

La puissance du DWH
fait le travail.

ELT est popularisé par les data warehouses cloud (BigQuery, Snowflake, Redshift) qui ont la puissance de transformer en SQL.

ETL vs ELT : quand utiliser quoi ?

Critère	ETL	ELT
Ordre	Extract → Transform → Load	Extract → Load → Transform
Où transformer ?	En mémoire (Python, Spark)	Dans la cible (SQL, dbt)
Volume	Petits à moyens volumes	Gros volumes (cloud)
Complexité transfo	Transformations complexes (ML, NLP)	Transformations SQL simples
Latence	Plus lent (transformation avant)	Plus rapide (données dispo vite)
Outils	Airflow, Luigi, Talend, Spark	dbt, Fivetran, Airbyte
Notre projet	✓ On utilise ETL	

Notre projet AniData Lab suit le pattern ETL : on extrait les CSV, on transforme en Python (nettoyage + features), puis on charge dans Elasticsearch.

Notre pipeline ETL — AniData Lab

Ce qu'on a fait manuellement hier, on va l'automatiser avec Airflow.



Le problème : c'était manuel !

Hier, vous avez lancé les scripts un par un dans le bon ordre. Mais en production, un pipeline doit tourner automatiquement, gérer les erreurs, retenter en cas d'échec, alerter quand ça plante, et s'exécuter selon un planning. C'est le rôle d'un orchestrateur.

Pourquoi un orchestrateur ?

Sans orchestrateur, un pipeline en production est un cauchemar.

Dépendances

Le script B dépend du script A. Si A échoue, B ne doit PAS se lancer. Avec un cron, B se lance quand même → crash.

Retries & erreurs

Un appel API échoue à cause d'un timeout réseau. Il faut retenter 3 fois avant d'abandonner et alerter.

Monitoring

Savoir en temps réel quel script tourne, lequel a échoué, depuis combien de temps, consulter les logs.

Scheduling

Lancer le pipeline tous les jours à 6h, ou toutes les heures, ou quand un fichier arrive.

Scalabilité

Paralléliser les tâches indépendantes, distribuer sur plusieurs machines, gérer les files d'attente.



CHAPITRE 2

Apache Airflow

L'orchestrateur open source le plus utilisé pour les pipelines de données.

Apache Airflow : c'est quoi ?

Apache Airflow est une plateforme open source d'orchestration de workflows. Créé par Airbnb en 2014, devenu projet Apache en 2019. Utilisé par Airbnb, Spotify, Twitter, Slack, Adobe, et des milliers d'entreprises.

Son principe : vous décrivez votre pipeline en Python (« ce qui doit tourner, dans quel ordre »), et Airflow se charge de l'exécuter, le monitorer et le relancer si besoin.

Pipelines as Code

Tout est écrit en Python. Pas d'interface drag & drop.
Versionnable avec Git.

Interface web

Dashboard pour suivre les exécutions, voir les logs, relancer les tâches.

Extensible

Des centaines d'opérateurs pour se connecter à tout (S3, GCP, Spark, ES...).

Airflow n'exécute pas les données lui-même — il orchestre les tâches qui les manipulent. C'est un chef d'orchestre, pas un musicien.

Architecture d'Airflow

4 composants qui travaillent ensemble.

Webserver

Interface web (port 8080).
Voir les DAGs, les logs,
relancer les tâches.

`http://localhost:8080`

Scheduler

Le cerveau. Décide quand
lancer chaque tâche selon
le planning et les dépendances.

`Tourne en permanence`

Executor

Exécute les tâches.
LocalExecutor (notre cas)
ou CeleryExecutor (distribué).

`LocalExecutor`

Métadatabase

PostgreSQL. Stocke l'état
des DAGs, tâches, variables,
connexions, historique.

`PostgreSQL`

Le vocabulaire Airflow

6 termes à maîtriser absolument.

DAG

Directed Acyclic Graph

Le pipeline. Un graphe de tâches avec un ordre d'exécution. « Acyclic » = pas de boucle infinie.

Task

Tâche

Une unité de travail dans un DAG. Ex : « charger le CSV », « nettoyer les données ».

Operator

Opérateur

Le type de tâche. PythonOperator, BashOperator, EmailOperator...

DAG Run

Exécution

Une instance d'exécution d'un DAG à une date donnée.

XCom

Cross-communication

Permet aux tâches de se passer des données entre elles.

Schedule

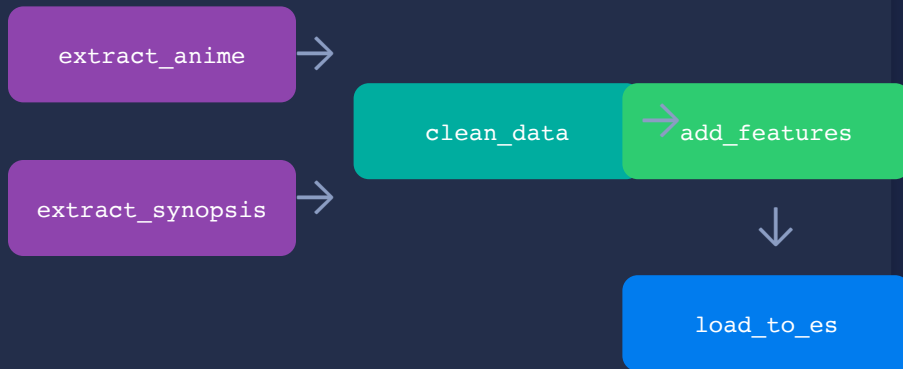
Planification

Quand le DAG doit tourner : @daily, @hourly, cron, ou None (manuel).

Le DAG : votre pipeline en graphe

DAG = Directed Acyclic Graph. Chaque nœud est une tâche, chaque flèche est une dépendance.

Exemple : pipeline AniData Lab



Directed

Les flèches ont un sens : A doit finir AVANT que B commence.

Acyclic

Pas de boucle : une tâche ne peut pas dépendre de son propre résultat.

Parallélisme

extract_anime et extract_synopsis tournent en parallèle !

Les Operators : les types de tâches

Chaque Operator sait faire UNE chose. Vous choisissez celui qui correspond à votre besoin.

PythonOperator

Exécute une fonction Python. Le plus utilisé. Notre choix principal.

BashOperator

Exécute une commande shell/bash. Utile pour lancer des scripts externes.

EmailOperator

Envoie un email. Utile pour les alertes en cas de succès ou d'échec.

BranchPythonOperator

Choisit quelle branche du DAG exécuter selon une condition Python.

TriggerDagRunOperator

Lance un autre DAG. Pour chaîner des pipelines.

Sensor (ex: FileSensor)

Attend qu'une condition soit remplie (fichier, heure, API...) avant de continuer.

Cycle de vie d'un DAG

De l'écriture du fichier Python à l'exécution des tâches.

- 1 Écriture**
Vous écrivez le fichier .py dans le dossier dags/
- 2 Parsing**
Airflow détecte le fichier et le parse (toutes les 30s)
- 3 Scheduling**
Le Scheduler décide quand lancer le DAG selon le schedule
- 4 DAG Run**
Une instance d'exécution est créée pour une date donnée
- 5 Task Instance**
Chaque tâche est exécutée par l'Executor dans l'ordre
- 6 Monitoring**
Résultat visible dans le Webserver : succès, échec, durée, logs



CHAPITRE 3

Écrire un DAG Airflow

La structure d'un fichier DAG en Python, pas à pas.

Structure de base d'un DAG

Tout DAG Airflow est un fichier Python dans le dossier dags/.

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator

default_args = {
    "owner": "anidata-lab",
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="mon_premier_dag",
    default_args=default_args,
    schedule_interval=None,          # Manuel
    start_date=datetime(2026, 3, 25),
    catchup=False,
) as dag:
    task_a = PythonOperator(task_id="task_a", python_callable=ma_fn)
    task_a >> task_b >> task_c
```

Les default_args : configuration des tâches

Appliqués à toutes les tâches du DAG (surchargeable par tâche).

owner

Qui est responsable du DAG. Apparaît dans l'UI.

retries

Nombre de tentatives en cas d'échec. 2 = réessaye 2 fois.

retry_delay

Temps d'attente entre chaque retry. timedelta(minutes=5).

email_on_failure

Envoyer un email si la tâche échoue. True/False.

depends_on_past

La tâche dépend-elle de la réussite de son exécution précédente ?

start_date

Date à partir de laquelle le DAG peut être schedulé. TOUJOURS dans le passé.

PythonOperator : exécuter du Python

L'Operator le plus utilisé. Il appelle une fonction Python comme une tâche.

```
def extraire_anime(**kwargs):
    """Tâche d'extraction du fichier anime.csv"""
    import pandas as pd
    df = pd.read_csv("/opt/airflow/data/anime.csv")
    print(f"✅ {len(df)} animes chargés")
    return len(df) # Retourné via XCom

# Créer la tâche
extract_task = PythonOperator(
    task_id="extraire_anime",
    python_callable=extraire_anime,
    provide_context=True, # Donne accès à **kwargs
)

# kwargs contient :
# - kwargs["ds"]           → date d'exécution ("2026-03-25")
# - kwargs["task_instance"] → accès aux XComs
# - kwargs["dag_run"]       → infos sur le run en cours
```

Définir l'ordre des tâches

L'opérateur >> (bitshift) définit les dépendances entre tâches.

```
# Séquentiel : A puis B puis C  
task_a >> task_b >> task_c
```

Séquentiel

```
# Fan-out (un vers plusieurs)  
task_a >> [task_b, task_c]
```

Fan-out

```
# Parallèle puis convergence  
[task_a, task_b] >> task_c
```

Parallèle + convergence

```
# Chaîne complète  
[extract_anime, extract_synopsis] \  
    >> clean >> features >> load
```

Chaîne complète (notre DAG)

XCom : passer des données entre tâches

XCom (cross-communication) permet aux tâches de s'échanger des petites données.

```
# Task A : PUSH une valeur
def task_a_func(**kwargs):
    nb_animes = 17562
    kwargs["ti"].xcom_push(key="count", value=nb_animes)
    # OU simplement : return nb_animes (auto-push)

# Task B : PULL la valeur de Task A
def task_b_func(**kwargs):
    count = kwargs["ti"].xcom_pull(task_ids="task_a", key="count")
    print(f"Task A a trouvé {count} animes")
```



XCom est fait pour des petites données (compteurs, chemins de fichiers, statuts). Pas pour passer un DataFrame entier ! Pour les gros volumes, écrivez dans un fichier et passez le chemin.

Scheduling : quand lancer le DAG ?

Le `schedule_interval` définit la fréquence d'exécution.

Schedule	Signification	Équivalent cron
None	Déclenchement manuel uniquement	—
@once	Une seule fois	—
@hourly	Toutes les heures	0 * * * *
@daily	Tous les jours à minuit	0 0 * * *
@weekly	Tous les lundis à minuit	0 0 * * 1
"0 6 * * *"	Tous les jours à 6h du matin	Cron custom

Pour notre projet, on utilise `schedule_interval=None` (déclenchement manuel). En production, on mettrait `@daily` pour rafraîchir les données tous les jours.

Gestion des erreurs et retries

Airflow relance automatiquement les tâches en échec selon votre configuration.

1

Retries automatiques

`retries=3 + retry_delay=5min` : la tâche est relancée 3 fois avec 5 min d'attente entre chaque.

2

Callbacks

`on_failure_callback` permet d'exécuter une fonction quand une tâche échoue (envoyer un Slack, un email...).

3

Timeout

`execution_timeout=timedelta(hours=1)` : la tâche est tuée si elle dépasse 1 heure.

4

Trigger rules

Par défaut : une tâche ne tourne que si TOUTES ses dépendances ont réussi. Modifiable.

```
# Exemple complet
task = PythonOperator(task_id="load", python_callable=load_fn,
                      retries=3, retry_delay=timedelta(minutes=2),
                      execution_timeout=timedelta(hours=1), on_failure_callback=alert_slack)
```



CHAPITRE 4

L'interface web Airflow

Naviguer dans le dashboard pour monitorer vos pipelines.

Les vues principales de l'UI

http://localhost:8080 — admin / admin

DAGs List

Vue d'ensemble de tous vos DAGs avec leur statut, dernier run, schedule. Toggle on/off pour activer.

Graph View

Visualisation du DAG sous forme de graphe. Les couleurs montrent le statut de chaque tâche : vert = succès, rouge = échec, jaune = en cours.

Tree View

Historique des runs sous forme de grille. Chaque colonne = un run, chaque ligne = une tâche.

Task Logs

Les logs de chaque tâche. Cliquer sur une tâche → Log. C'est là qu'on debug quand ça plante !

Gantt View

Timeline d'exécution. Montre la durée de chaque tâche et le parallélisme. Idéal pour optimiser.

Les couleurs de statut dans Airflow

Chaque tâche dans l'UI a une couleur qui indique son état.



success

La tâche s'est terminée avec succès.



failed

La tâche a échoué (après tous les retries).



running

La tâche est en cours d'exécution.



up_for_retry

La tâche a échoué, en attente de retry.



queued

La tâche est en file d'attente.



no_status

La tâche n'a pas encore été exécutée.



upstream_failed

Une tâche en amont a échoué — celle-ci est annulée.



skipped

La tâche a été sautée (branchement conditionnel).



CHAPITRE 5

Bonnes pratiques

Les règles d'or pour des DAGs robustes en production.

Les 8 règles d'or

Idempotence

Un DAG exécuté 2 fois produit le même résultat. Pas d'effets de bord.

Atomicité

Chaque tâche fait UNE chose. Si elle échoue, on sait immédiatement où.

Pas de logique dans le DAG

Le fichier DAG définit la structure, pas le code métier. Le code va dans des modules séparés.

catchup=False

Sauf besoin explicite, désactivez le rattrapage pour éviter des dizaines de runs au démarrage.

Nommage explicite

dag_id et task_id clairs : « etl_anime_extract », pas « task_1 ».

XCom = petit volume

Passez des chemins de fichiers, pas des DataFrames. XCom est stocké en base.

Logs abondants

print() dans vos fonctions = logs visibles dans l'UI. Loguez les compteurs et étapes.

Tester localement d'abord

Testez vos fonctions Python en dehors d'Airflow avant de les wrapper dans un Operator.

Synthèse & prochaines étapes

ETL = Extract → Transform → Load (transformation avant chargement).

ELT = Extract → Load → Transform (transformation dans la cible).

Airflow orchestre les pipelines : DAG, Task, Operator, Schedule.

Un DAG est un graphe de tâches avec des dépendances, écrit en Python.

PythonOperator est votre outil principal pour exécuter du code.

→ Place à la pratique : vous allez créer votre premier DAG Airflow qui automatise le pipeline AniData Lab de bout en bout (Extract → Transform → Load dans ES) !